

```
#include <xc.h> // Bibliothèque pour PIC  
(XC8)
```

```
// Configuration des bits (exemple pour un  
PIC16F877A)
```

```
#pragma config FOSC = HS // Oscillateur  
haute fréquence
```

```
#pragma config WDTE = OFF //  
Watchdog Timer désactivé
```

```
#pragma config PWRTE = OFF // Power-  
up Timer désactivé
```

```
#pragma config BOREN = ON // Brown-  
out Reset activé
```

```
#pragma config LVP = OFF // Low-  
Voltage Programming désactivé
```

```
#pragma config CPD = OFF // Protection  
EEPROM désactivée
```

```
#pragma config WRT = OFF // Protection  
écriture désactivée
```

```
#pragma config CP = OFF // Protection  
code désactivée
```

```
#define _XTAL_FREQ 20000000 // 20 MHz  
(à adapter selon ton quartz)
```

```
#include <stdint.h> // pour uint8_t
```

```
// Tableau de correspondance segments  
pour 0-9
```

```
uint8_t segments[10] = {
```

```
    0x3F, // 0
```

```
    0x06, // 1
```

```
    0x5B, // 2
```

```
    0x4F, // 3
```

```
    0x66, // 4
```

```
    0x6D, // 5
```

```
    0x7D, // 6
```

```
    0x07, // 7
```

```
    0x7F, // 8
```

```
    0x6F  // 9
```

```
};
```

// Fonction : envoyer la valeur aux segments (exemple sur PORTD)

```
void afficher_segment(uint8_t  
valeur_segment) {  
    PORTD = valeur_segment;  
}
```

// Fonction : activer l'afficheur unités, dizaines, centaines (exemple sur PORTB)

```
void activer_afficheur(uint8_t position) {  
    switch(position) {  
        case 0: // unités  
            PORTB = 0x01; // RB0 actif  
            break;  
        case 1: // dizaines  
            PORTB = 0x02; // RB1 actif  
            break;  
        case 2: // centaines  
            PORTB = 0x04; // RB2 actif  
            break;  
    }
```

```
}
```

```
// Délai en millisecondes
```

```
void delay_ms(int ms) {
```

```
    while(ms--) {
```

```
        __delay_ms(1); // fonction XC8 : délai
```

```
1 ms
```

```
    }
```

```
}
```

```
void afficher_nombre(int nombre) {
```

```
    int unité = nombre % 10;
```

```
    int dizaine = (nombre / 10) % 10;
```

```
    int centaine = (nombre / 100) % 10;
```

```
    activer_afficheur(2); // centaines
```

```
    afficher_segment(segments[centaine]);
```

```
    delay_ms(5);
```

```
    activer_afficheur(1); // dizaines
```

```
    afficher_segment(segments[dizaine]);
```

```
    delay_ms(5);

    activer_afficheur(0); // unités
    afficher_segment(segments[unité]);
    delay_ms(5);
}

void main(void) {
    // Initialisation
    TRISD = 0x00; // PORTD en sortie
    (segments)
    TRISB = 0x00; // PORTB en sortie
    (sélection afficheurs)
    PORTD = 0x00; // Éteindre segments
    PORTB = 0x00; // Désactiver afficheurs

    while (1) {
        for (int i = 0; i <= 999; i++) {
            for (int t = 0; t < 50; t++) { // afficher
plusieurs fois pour lisibilité
                afficher_nombre(i);
            }
        }
    }
}
```

}

}

}

}